

Software Design Document

AI Task Agent Using More Flexible Generic Tools via Model Context Protocol (MCP)

Team Members

Jadon Duff, Computer Science (jduff2024@my.fit.edu)

Jacob Ewasko, Computer Science (jewasko2024@my.fit.edu)

Ioannis Limniatis, Computer Science (ilimniatis2023@my.fit.edu)

Jacob Lebkuecher, Software Engineering (jlebkuecher2023@my.fit.edu)

Faculty Advisor

Eraldo Ribeiro, College of Engineering and Science: Department of Electrical Engineering and
Computer Science (eribeiro@fit.edu)

Client

Doug Armstrong, District Directorate Chief - Response

of

The United States Coast Guard Auxiliary

Table of Contents

1. Introduction
 - 1.1. Purpose
 - 1.2. Scope
 - 1.3. Overview
 - 1.4. Definitions
2. System Overview
3. System Architecture
 - 3.1. Architectural Design
 - 3.2. Decomposition Description
 - 3.3. Design Rationale
4. Data Design
 - 4.1. Data Description
 - 4.2. Data Dictionary
5. Component Design
6. Human Interface Design
 - 6.1. Overview of User Interface
 - 6.2. Screen Images
 - 6.3. Screen Objects and Actions
7. Requirements Matrix

1.0 Introduction

1.1 Purpose

This document serves as the Software Design Document for the AI Task Agent Using More Flexible Generic Tools via Model Context Protocol (MCP) project (ATA). It describes the architecture and system design of the ATA. The ATA is an agentic system that encompasses (a) a fine-tuned LLM, (b) a deployment environment/HELM chart, (c) a Model Context Protocol (MCP) server, (d) a virtual machine (VM), (e) AI memory management, encompassing Retrieval-Augmented Generation (RAG) and context window management, and (f) a graphical user interface (GUI).

1.2 Scope

Currently, MCP servers are tuned to a single domain and preprogrammed tools. The ATA will design an MCP server with tools dynamic enough to allow an AI Agent to execute most human-like actions on a VM, resolving this issue.

LLMs are also constrained by their cutoff window (memory) and tuned for general versatility. This project will provide an automated manner to tune an LLM for a specific use case. The ATA will also design a system to handle dynamic context windows and AI memory management to allow the AI Agent to work with large systems.

Finally, a multitude of systems and APIs can lead to security concerns and vulnerabilities. This project will allow for (a) siloed deployments (e.g. each subsystem separately, matching current “more vulnerable” systems), and (b) secure air-gapped deployments (all subsystems, including LLM, local).

The uniqueness of this project is that, while some of these individual subproblems have been attempted in industry; some have not been solved disparately, others have not been solved together, and the system has not been packed for dynamic deployment.

1.3 Overview

This document will first provide an overview of the ATA system, including a general description of its functionality, context, and design.

The document will then outline the architecture of the ATA, including a high-level overview of its responsibilities; how they are partitioned and assigned to subsystems; how they collaborate to achieve desired functionality; a decomposition of its subsystems; and, finally, the rationale for selecting the architecture, including critical issues and trade-offs.

This is followed by an explanation of how the ATA's information domains were mapped to data structures, along with a list of the system entities, their types, and descriptions.

Then the document will examine the design of the ATA's components more systematically, summarizing their algorithms and object member functions.

After that, the document will showcase the human interface design, describe system functionality from the user's perspective and how users will use the system, and provide graphics of the user interface, followed by a short explanation of screen objects and their actions.

Finally, the document will provide a cross-reference linking the ATA's components and data structures to the requirements in the software requirements specification (SWRS). It will show which system components satisfy each functional requirement in the SWRS.

2.0 System Overview

Often, various popular Large Language Models (LLMs) can reason through and complete various given tasks. However, it is often limited to either the website itself or can only provide text and simple downloadable files. To act in a more complex matter, LLM's can be connected via tools; the MCP (Model Context Protocol) has been created as a result. However, MCPs are still flawed and only built for single domains,

The AI Task Agent will be an air gapped MCP server utilizes various simple tools including (executing shell commands, reading and writing files, capturing the screen, and sending keyboard and mouse input] to let the agent control and perform actions a human could for convenience, or for immediate query. These actions are carried out in a sandboxed virtual machine (VM), so the agent can work freely without risking or damaging the host system.

From the user's perspective, the AI Task Agent will work in these following steps. The user that deployed the task agent will be able to describe a task in natural language through the Client GUI. The agent will then evaluate the given task, send the given task along with relevant context retrieved from its memory, to the LLM. The LLM decides the action and issues a tool will call, which the MCP server executes on the VM. The result will be returned to the LLM which will decide the next action. This cycle repeats until the given task is completed. Progress will be visible throughout.

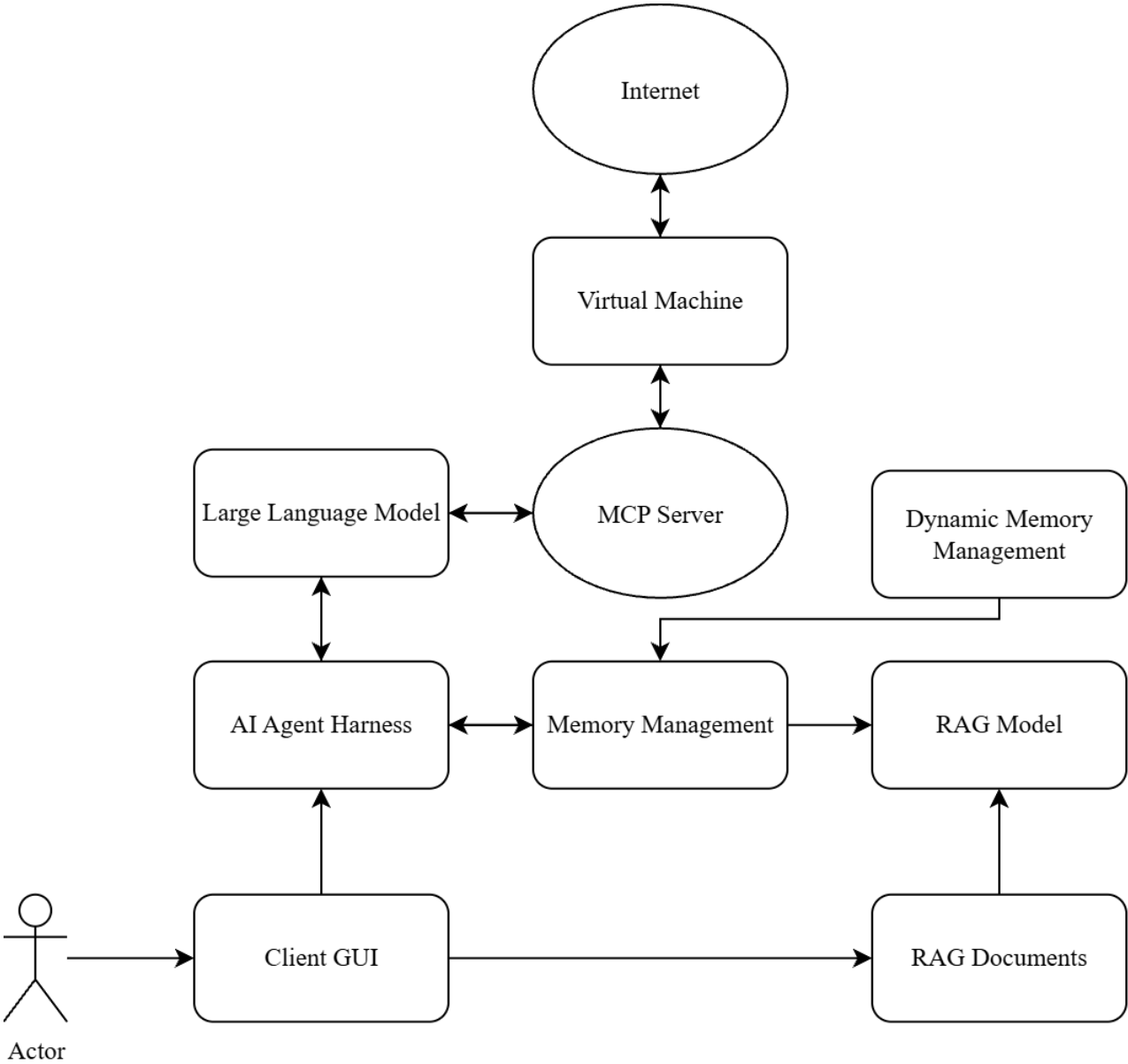
The agent will work with large systems and domain-specific and document specific information, and as such it will be a memory management subsystem. Retrieval-Augmented Generation (RAG) lets users upload reference documents that the agents can use and study as references. It will also focus on context window management to decide what information stays in the LLM's every growing context if the task were to continue to grow.

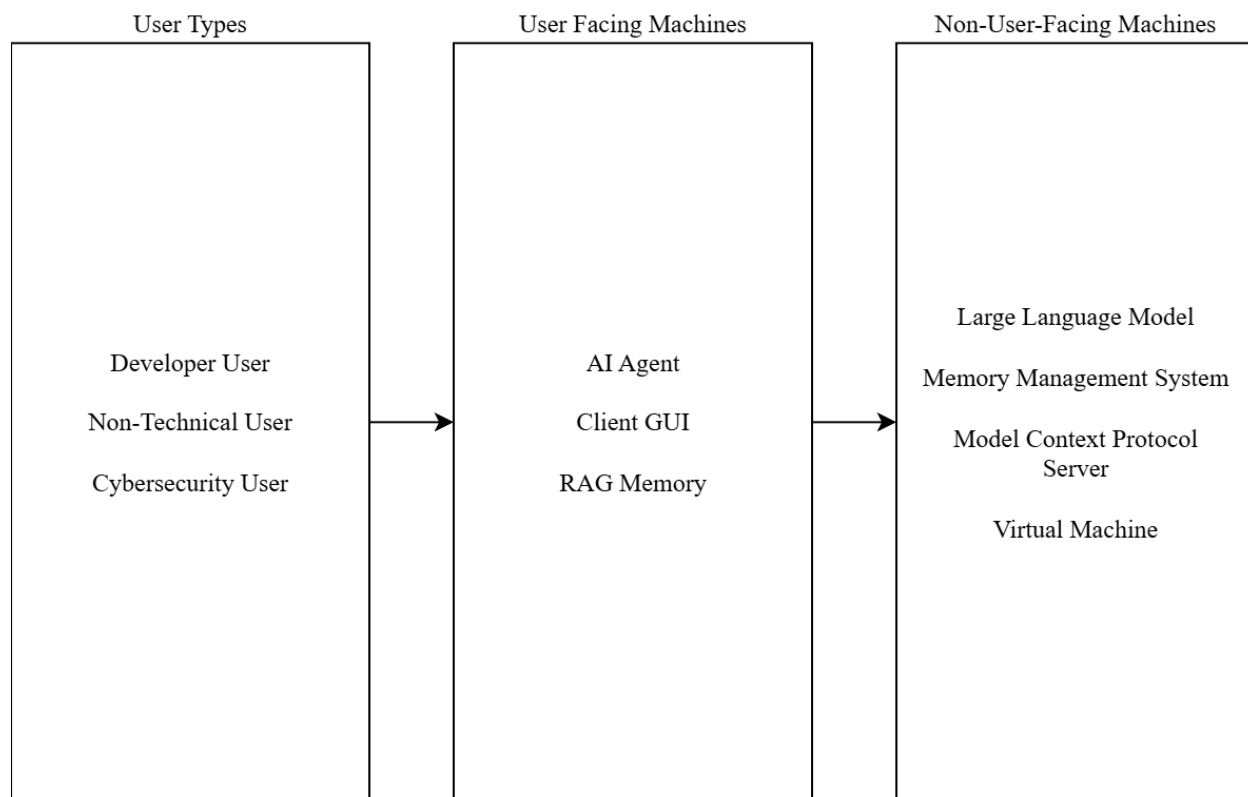
The AI Task Agent is designed for easy access and use. Non-technical users should be able to easily ask and use it no matter the context, while developers too can fine tune and use the system to achieve proper goals.

3.0 System Architecture

3.1 Architectural Design

The AI Task Agent is decomposed into several independent subsystems, each deployed in its own sandboxed service. This modular structure is what makes both siloed and air-gapped deployments possible: each subsystem can be placed on separate infrastructure, or can all be deployed together in a singular cluster. The below figures show the major subsystems and their interconnections and how they interact.





3.2 Decomposition Description

Looking at the functional description of the system, Figure 3.1.A shows the general structural decomposition of our Agent, while Figure 3.1.B shows the top-level data flow.

All three user types: The developer, non-technical, and cybersecurity user, interact with the system through the Client GUI. The GUI sends tasks requests to the AI Agent Harness and sends the uploaded files to the RAG Documents, where the RAG Model will then index them for future retrieval. The harness is generally the central controller. It gathers context from Memory Management, sends the request to the LLM (Large Language Model), and returns the results back to the GUI. Memory Management stores the conversation and task state to pull the most relevant document passages from the RAG Model. Dynamic Memory Management then decides what to keep, summarize, or discard, so the LLM stays within context limits.

When the LLM needs to act, it issues a call to the MCP Server; the server carries out the action inside the sandboxed Virtual Machine, an isolated Linux environment where commands are run in a safe environment. The VM is the only system with access to the internet, but the link is in air-gapped deployments. Results are passed back to the MCP server and to the LLM. The loop repeats until the task is complete, and the harness is then updated to save the state.

3.3 Design Rationale

The reason we have designed a more sandboxed system that relies on virtual environments is because the AI Task Agent, if used wrongly, could potentially damage or

delete important files on the host machine. By sandboxing it in a Linux environment, every command or directive done can be done barred from the host's actual machine, avoiding potential damage under the worst case.

Routing every action through the MCP server gives the LLM a proper and consistent standard interface for tools and can keep it from touching the execution environment, so no particularly harmful executions can occur. We air-gapped internet links to keep access moderate and safe.

4.0 Data Design

4.1 Data Description

User Input is one of the main information domains within the ATA. It is what the system pulls from to create tasks. Once the user selects send the user's input is immediately sent to the AI Agent harness. Here the user's prompt is broken down and transformed into an agent capable of executing these multi-step tasks. This information is stored within memory if needed short-term or stored in a database/locally for long-term recall.

RAG Documentation is another one of the main domains of information within the ATA. It's what the RAG models pull from (especially in an air-gapped system) before an AI agent answers a prompt. The ATA parses through the RAG documentation using RAG models to index, retrieve, and incorporate the text from the supplied documentation into its output. It is stored within a RAG database locally on the user's machine and pulled from whenever it is needed by the RAG models. There are two types of RAG documentation, reliable and relevant documentation. Reliable documentation are documents provided and curated by the user as they are a reliable source for the RAG to pull from. Relevant documentation are documents filled with search-by-keywords that are determined by the model. These documents are used to search against the RAG in order to pick a relevant subset.

External Information is the largest information domain the ATA will be using. It consists of all information available to the system outside of itself. This includes information on the internet, system information, system files, user information, etc. This information is grabbed from external sources using an MCP server hooked up to the LLM. The LLM interfaces with an MCP server, using this information to supplement the current task. External information is required for all tasks as its information that must be grabbed from outside the VM. Without this form of information, the ATA would be unable to perform any tasks outside the sandbox. Moreover, external information is one of the most powerful aspects of the ATA because of the things it can help do. For instance, with an MCP server, the LLM can ingress information from a repository and send it to the LLM where it then uses this information to perform a task like a pull request or a commit.

5.0 Component Design

This overview will go into detail about the several components found within our design for the AI Task Agent.

Client GUI: The Client GUI is the main workspace for the user, where they will type and provide a task to the agent, as well as providing any supplementary documentation that would assist in the context of the task. Documents that are verified and accepted are given to the RAG, and the task provided will go to the Agent Harness.

Rag Documents and Model: RAG Documents store the files and documentation the user uploads. The RAG model extracts the document's text and splits it into smaller chunks. These chunks are converted into a vector stored in a searchable index along with the name of the source file. When Memory Management sends a query, the text is vectorized and compared with the information that has also been vectorized.

Memory Management: Keeps a constantly running history for each task, starting from the initialization of the conversation, every time the LLM responds and how each tool responds and provides a result. The harness will need context for the next step, so Memory Management combines relevant task history and passes the context over to Dynamic Memory Management to stay within context limit.

Dynamic Memory Management: Decides what stays in the LLM's context window once there is too much history. Ranks items by relevance and recency and keeps the highest-ranked ones until it reaches a token limit. This is to ensure that rate limit is controlled and not wasted under larger tasks. Older history is often summarized to keep it short and concise; any leftovers are dropped to make space.

MCP Server: The MCP server gives the LLM a fixed set of generic tools, including running shell commands, reading files, writing files, capturing the screen, and sending keyboard and mouse inputs. When receiving a tool call, it will always verify that the tools are available to be used, rejecting any garbage data. It forwards the action to the Virtual Machine and returns the result to the LLM.

Virtual Machine: The Virtual Machine is a sandboxed, isolated Linux environment where every tool action actually runs. Things like shell commands can be run without impacting the host machine. Internet Access is controlled in air-gapped deployments, so all outside requests are blocked. If the environment ends up in a bad state, the environment can be restored, and the user is notified, so no mistake is permanent.

Internet: The Internet is an external resource only reached from the Virtual Machine, and only when the deployment and user allow it to access the internet. The internet is not available on air-gapped systems.

6.0 Human Interface Design

6.1 Overview of User Interface

The main interface is where the user will be doing most of their interactions. The central box is where the user will input text related to tasks, attach attachments like files, images, and videos, and select the model to use. Here the user can also

see the current air-gapped status and shut down the ATA in the case of emergency.

To the left is where the user can view tasks and manage them. Here they can rename, export, and delete tasks. They can also search for and view their task history. To the right is where the user can view active tasks and past tasks. The user can select the terminate option to terminate a currently active task.

At the bottom of the task list, the user can select the settings option to go to settings. The settings interface is where the user can change the settings of the ATA. This includes options to uninstall the ATA, convert it to air-gapped (only if currently networked), and upload a GGUF if they wish. Here there are re-installation options as well, so if the user would like to switch from air-gapped to networked or to include tools not currently installed.

The RAG documentation menu is where the user will view and manage RAG documentation. The user can add new documents and search through the knowledge base. The user can select documents from the list to view them from the left side of the screen; a document can be deleted from the list too.

6.2 Screen Images

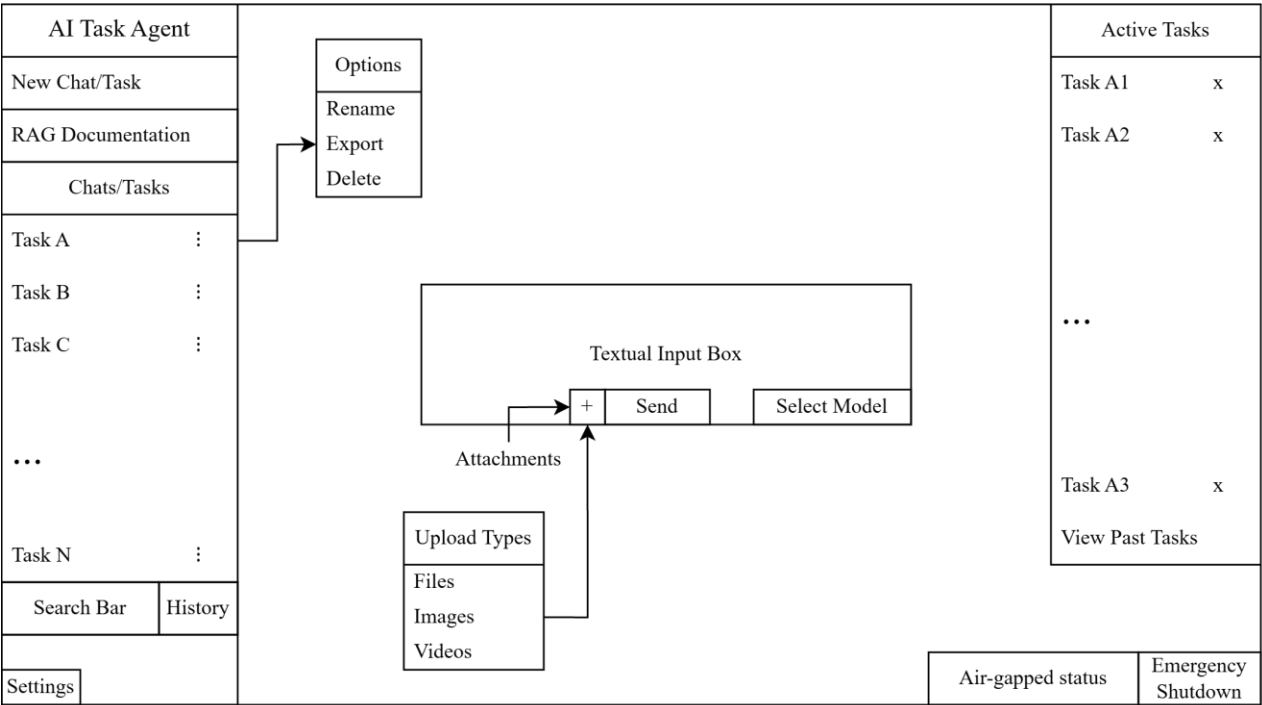


Figure 6.2a: Main Interface

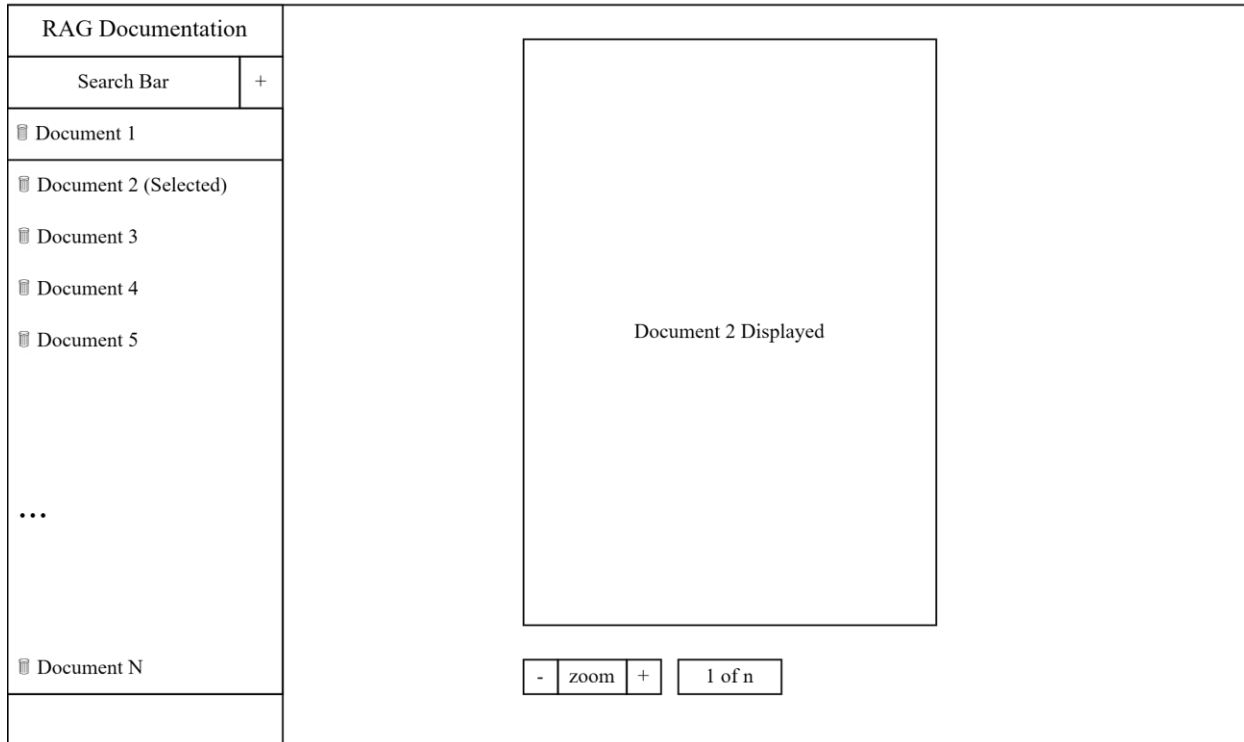


Figure 6.2b: RAG Documentation Interface

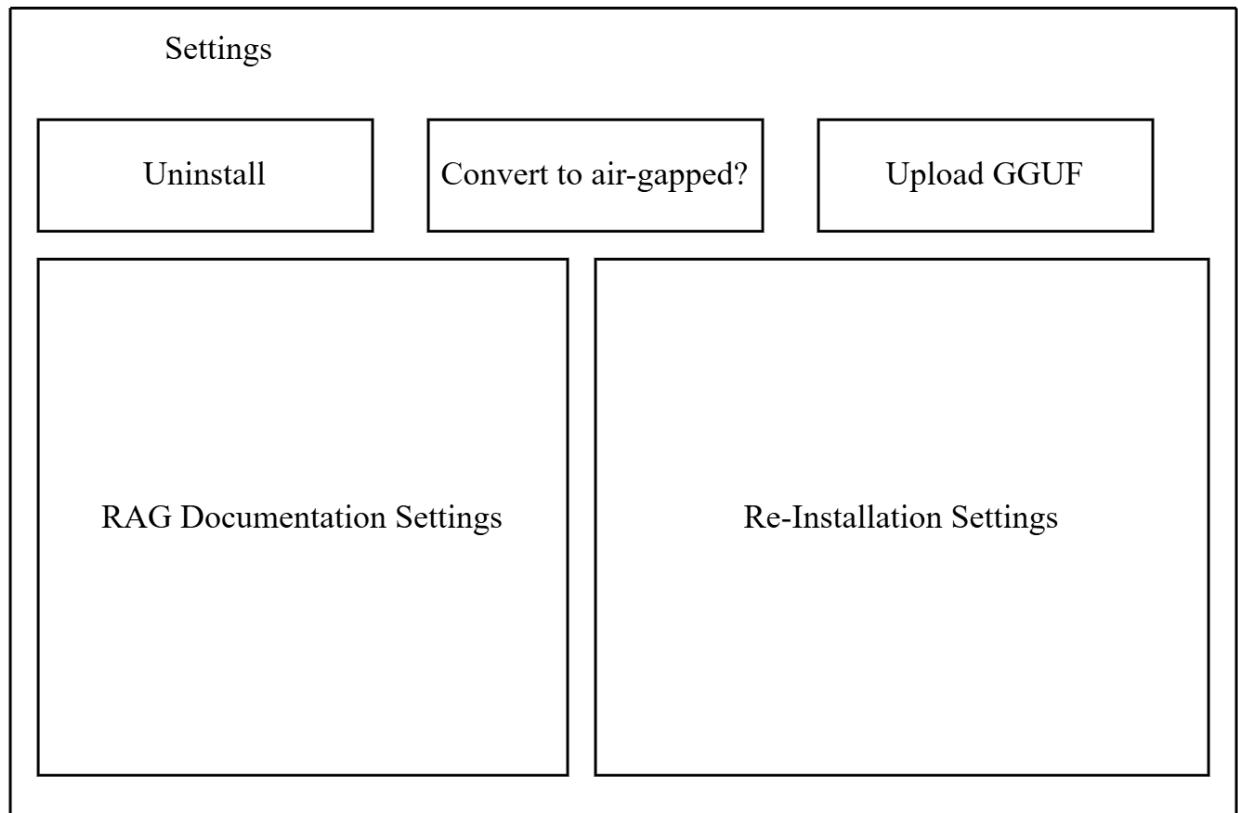


Figure 6.2c: ATA settings interface.

6.3 Screen Objects and Actions

The software from the user perspective will feature a central container. This container allows the user to input text for tasks, attach files, choose different models, and send messages. When selecting attach files, options for media types show up, including files, videos, and images. At the bottom righthand corner of this main interface, the user can view the current air-gapped status, as well as activate emergency shutdown if necessary.

On the left-hand side of the screen is where a large part of the functionality of the system is held. There is an option to create a task and below is a list of all current and previous tasks. The user can search through their tasks via a search bar or by scrolling down. Eventually, they may be unable to keep scrolling and can click the history tab to see a full history of tasks located at the bottom of the screen. By clicking on a task, the user can view the task's history, including every message sent. They can select more options on the right side of a singular task. This shows any options related to a task itself such as renaming, exporting, and deleting tasks. At the bottom of the task list is the search bar. This allows the user to search through their tasks.

Below the search bar and history buttons are the settings button. Once selected, a container is shown, allowing the user to change any settings related to the ATA. These settings include the option to fully uninstall the ATA, conversion to air-gapped (only from networked to air-gapped). For the inverse, there are re-installation settings that the user can use to re-install the ATA to include any tools or other parameters related to installation.

Within this left-hand section, the user can select a button to manage RAG documentation. Once selected, the user is brought to an interface allowing them to view and manage RAG documentation. It is similar to the task interface; the user can view the RAG documents on the left-hand side. Above this list the user can select + to add a document to the knowledge base or search through it via a search bar. On the right-hand side of each document is the option to trash it if necessary. Once the user selects a document, it will be displayed within the center of the screen. If the user scrolls up and down, they can scroll through the pages of the document like any other document viewer. At the bottom of the document display, there are buttons for the user to move between pages and zoom in or out if needed.

Moving back to the main interface, if the looks to the right-side, they can see what tasks are currently active. They can scroll through them if there are more than the current container can visualize. At the bottom of the list, the user can select past tasks, allowing them to see past tasks. The user can click the terminate button, located to the right of a task in the active tasks list. Once selected, the task will terminate, stopping it from continuing.

7.0 Requirements Matrix

Component	Requirements (IDs) Satisfied
<i>Client GUI</i>	<i>ATA-FTE-001, ATA-FTE-007, ATA-FTE-008, ATA-FAI-002, ATA-FAI-005, ATA-FAI-006, ATA-FAI-007, ATA-FAI-008, ATA-FAI-009, ATA-FDP-002, ATA-FDP-005, ATA-FDP-008, ATA-FDP-014, ATA-FDP-017, ATA-FDP-020, ATA-IUI-001, ATA-IUI-002, ATA-IUI-003, ATA-IUI-004, ATA-IUI-005, ATA-IUI-006, ATA-IUI-007, ATA-IUI-008, ATA-IUI-009, ATA-IUI-010, ATA-IUI-011, ATA-IUI-012, ATA-IUI-013, ATA-IUI-014, ATA-IUI-015, ATA-IUI-016, ATA-ISI-002</i>
<i>RAG Documents</i>	<i>ATA-FTE-032, ATA-FAI-001, ATA-FAI-002, ATA-IUI-005, ATA-IUI-016, ATA-OPS-024</i>
<i>RAG Models</i>	<i>ATA-FTE-032, ATA-FTE-037, ATA-FTE-037, ATA-FAI-001, ATA-FAI-002, ATA-IUI-005, ATA-OPS-013, ATA-OPS-014, ATA-OPS-024</i>
<i>Memory Management</i>	<i>ATA-FTE-019, ATA-FTE-021, ATA-FTE-022, ATA-FTE-023, ATA-FTE-024, ATA-FTE-031, ATA-FTE-035, ATA-FTE-037, ATA-FAI-005, ATA-IUI-005, ATA-FAI-006, ATA-FAI-007, ATA-IUI-008, ATA-IUI-009, ATA-IUI-013, ATA-OPS-008</i>
<i>AI Agent Harness</i>	<i>ATA-FTE-001, ATA-FTE-002, ATA-FTE-005, ATA-FTE-009, ATA-FTE-010, ATA-FTE-011, ATA-FTE-012, ATA-FTE-013, ATA-FTE-014, ATA-FTE-015, ATA-FTE-016, ATA-FTE-017, ATA-FTE-018, ATA-FTE-020, ATA-FTE-022, ATA-FTE-023, ATA-FTE-024, ATA-FTE-025, ATA-FTE-027, ATA-FTE-028, ATA-FTE-029, ATA-FTE-030, ATA-FTE-038, ATA-FTE-039, ATA-FAI-003, ATA-OPS-001, ATA-OPS-003, ATA-OPS-005, ATA-OPS-006, ATA-OPS-007, ATA-OPS-008, ATA-OPS-009, ATA-OPS-010, ATA-OPS-011, ATA-OPS-012, ATA-OPS-013, ATA-OPS-014, ATA-OPS-015, ATA-OPS-016, ATA-OPS-017, ATA-OPS-018, ATA-OPS-019, ATA-OPS-021, ATA-OPS-022, ATA-OPS-023, ATA-OPS-025, ATA-OPS-026, ATA-OPS-027</i>
<i>Large Language Model (LLM)</i>	<i>ATA-FTE-001, ATA-FTE-009, ATA-FTE-010, ATA-FTE-012, ATA-FTE-013, ATA-FTE-014, ATA-FTE-015, ATA-FTE-016, ATA-FTE-017, ATA-FTE-018, ATA-FTE-020, ATA-FTE-022, ATA-FTE-023, ATA-FTE-024, ATA-FTE-025, ATA-FTE-027, ATA-FTE-028, ATA-FTE-029,</i>

	ATA-FTE-030, ATA-FTE-031, ATA-FTE-032, ATA-FTE-033, ATA-FTE-038, ATA-FTE-039, ATA-FAI-003, ATA-FAI-009, ATA-IUI-010, ATA-PFR-005, ATA-OPS-001, ATA-OPS- 002, ATA-OPS-003, ATA-OPS-004, ATA- OPS-005, ATA-OPS-006, ATA-OPS-007, ATA-OPS-008, ATA-OPS-009, ATA-OPS- 010, ATA-OPS-011, ATA-OPS-012, ATA- OPS-013, ATA-OPS-014, ATA-OPS-015, ATA-OPS-016, ATA-OPS-017, ATA-OPS-018, ATA-OPS-021, ATA-OPS-022, ATA-OPS-023, ATA-OPS-024, ATA-OPS-025, ATA-OPS- 026, ATA-OPS-027, ATA-OPS-028
<i>Dynamic Memory Management</i>	ATA-FTE-019, ATA-FTE-020, ATA-FTE-021, ATA-FTE-022, ATA-FTE-023, ATA-FTE-033, ATA-FAI-003, ATA-FAI-004, ATA-FAI-006, ATA-FAI-007, ATA-FAI-008, ATA-OPS-019, ATA-OPS-020
<i>MCP Server</i>	ATA-FTE-002, ATA-FTE-006, ATA-FTE-007, ATA-FTE-008, ATA-FTE-010, ATA-FTE-011, ATA-FTE-012, ATA-FTE-016, ATA-FTE-017, ATA-FTE-018, ATA-FTE-020, ATA-FTE-021, ATA-FTE-022, ATA-FTE-023, ATA-FTE-024, ATA-FTE-025, ATA-FTE-027, ATA-FTE-028, ATA-FTE-029, ATA-FTE-030, ATA-FTE-031, ATA-FTE-032, ATA-FTE-033, ATA-FTE-034, ATA-FTE-035, ATA-FTE-036, ATA-FTE-038, ATA-FTE-039, ATA-FAI-003, ATA-FDP-002, ATA-FDP-008, ATA-IUI-012, ATA-ISI-001, ATA-ISI-002, ATA-OPS-003, ATA-OPS-004, ATA-OPS-005, ATA-OPS-006, ATA-OPS- 007, ATA-OPS-008, ATA-OPS-009, ATA- OPS-010, ATA-OPS-011, ATA-OPS-012, ATA-OPS-013, ATA-OPS-015, ATA-OPS- 016, ATA-OPS-017, ATA-OPS-018, ATA- OPS-021, ATA-OPS-024
<i>Virtual Machine</i>	ATA-FTE-003, ATA-FTE-004, ATA-FTE-009, ATA-FTE-011, ATA-FTE-012, ATA-FTE-013, ATA-FTE-014, ATA-FTE-015, ATA-FTE-016, ATA-FTE-017, ATA-FTE-018, ATA-FTE-019, ATA-FTE-020, ATA-FTE-023, ATA-FTE-031, ATA-FTE-036, ATA-FTE-037, ATA-FTE-038, ATA-FDP-001, ATA-IUI-007, ATA-IUI-012, ATA-ISI-004, ATA-OPS-007
<i>Internet</i>	ATA-FTE-023, ATA-FTE-025, ATA-FTE-026, ATA-FTE-027, ATA-FTE-028, ATA-FTE-029,

	<i>ATA-FTE-030, ATA-ISI-001, ATA-ISI-002, ATA-ISI-003, ATA-ISI-005, ATA-PFR-008, ATA-OPS-012, ATA-OPS-015, ATA-OPS- 016, ATA-OPS-017, ATA-OPS-018, ATA- OPS-021, ATA-OPS-022</i>
--	--